Uma expressão lambda em Java é uma função anônima (sem nome, sem tipo de retorno explícito e sem modificador de acesso) que permite escrever código de forma mais curta e direta. [1, 2, 3]

Elas foram introduzidas no **Java 8** para trazer conceitos de programação funcional à linguagem e reduzir a necessidade de criar classes anônimas ou várias linhas de código para tarefas simples. [1, 2, 3]

Principais Características

- **Função Anônima:** Não tem um nome próprio. [1]
- **Baseada em Interfaces Funcionais:** Só funciona com interfaces que possuem **exatamente um único método abstrato** (muitas vezes marcadas com a anotação `@FunctionalInterface`). [1, 2]
- **Sintaxe com Seta (`->`):** É dividida em três partes principais:
  A. **Lista de parâmetros** (à esquerda da seta)
  B. **Operador seta** (`->`)
  C. **Corpo da expressão** (à esquerda ou entre chaves `{}` à direita) [1, 2, 3]

Exemplo Prático

Sem usar lambda (com classe anônima tradicional):

java

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Olá sem lambda!");
    }
};
```

Use o código com cuidado.

Com o uso de expressão lambda:

java

```
Runnable r = () -> System.out.println("Olá com lambda!");
```

Use o código com cuidado.

Uso com a Stream API

As lambdas são muito usadas em conjunto com a **Stream API** para filtrar, transformar e manipular listas e coleções de dados: [1, 2]

java

```java
List<String> nomes = List.of("Ana", "Bruno", "Carlos");

nomes.stream()
     .filter(nome -> nome.startsWith("A"))
     .forEach(System.out::println); // Imprime "Ana"
```

Use o código com cuidado.

Saiba mais detalhes no artigo explicativo da [DevMedia](https://www.deviando... -> https://www.devmedia.com.br/como-usar-funcoes-lambda-em-java/32826) ou veja dicas práticas no blog da TreinaWeb. [1, 2]